

03 — GP Regression and Inference

What you'll learn: the exact GP regression algorithm step by step, the Cholesky-based computation, where the $O(n^3)$ cost comes from, and practical issues (noise, jitter, predictive covariance).

Prereqs: 01, 02.

Why it matters here: this is literally what `GpDvlVelocityModel.fit() / .predict()` will run, and the cost analysis is why edge needs docs 05/08.

1. The exact regression algorithm

Given training inputs X , targets y , noise σ_n^2 , kernel k , and test inputs X^* :

1. Build the training covariance $K = K(X, X)$ ($n \times n$) and add noise: $K_y = K + \sigma_n^2 I$.
2. **Cholesky factorize** $K_y = L L^T$ (L lower-triangular). This is the expensive step.
3. Solve for α : $\alpha = L^T \setminus (L \setminus y)$ (two triangular solves). α encodes the fitted weights.
4. Predictive mean: $\text{mean}^* = K(X^*, X) \cdot \alpha$.
5. Predictive covariance: solve $v = L \setminus K(X, X^*)$, then $\text{cov}^* = K(X^*, X^*) - v^T v$.
6. (Training) Log marginal likelihood: $-\frac{1}{2} y^T \alpha - \sum \log \text{diag}(L) - (n/2) \log 2\pi$.

Steps 4–5 are the two outputs we care about; step 6 is what hyperparameter optimization maximizes.

2. Why Cholesky (not a raw inverse)

We never form K_y^{-1} explicitly. Cholesky + triangular solves are faster and far more numerically

stable than computing an inverse. The factor L is reused for the mean, the covariance, and the marginal likelihood (its log-determinant is just $2 \sum \log \text{diag}(L)$). This is the standard, robust recipe; GPyTorch does the equivalent (with extra tricks) under the hood.

3. The cost — and why it dominates everything

- **Time:** the Cholesky factorization of an $n \times n$ matrix is $O(n^3)$.
- **Memory:** storing K is $O(n^2)$.

At $n = 1,000$ that's $\sim 10^9$ flops per fit and ~ 8 MB; at $n = 10,000$ it's $\sim 10^{12}$ flops and ~ 800 MB —

infeasible to refit at sensor rate, and hopeless on an embedded vehicle CPU. This single fact drives the entire architecture:

- For the demo we keep n small with a **sliding window** of recent DVL samples (e.g. last 60–180 s

at ~ 1 Hz $\rightarrow n \approx 60\text{--}180$), so $O(n^3)$ is trivial.

- For edge we switch to the **state-space/Kalman** form, which is $O(n)$ exact for temporal data

(doc 05), or to **sparse/inducing-point** approximations (doc 08).

4. Observation noise σ_n^2 and heteroscedastic noise

The $\sigma_n^2 \mathbf{I}$ term models sensor noise. Two refinements matter for us:

- **Per-sample noise.** Our DVL measurements can carry their own covariance (the repo's `RawDvlMeasurement.velocity_covariance_dvl / CorrectedDvlMeasurement.covariance_body`). Replace the scalar $\sigma_n^2 \mathbf{I}$ with a **diagonal** `Σ_noise`

built from per-sample variances so noisy samples are trusted less. This is standard and supported

by GP libraries.

- **Heteroscedasticity.** If noise varies with operating condition (altitude, beam count), the noise itself can be modelled — out of scope for the first build but noted.

5. Jitter / numerical conditioning

If two inputs are nearly identical or the kernel is very smooth, K_y becomes near-singular and Cholesky fails. The fix is **jitter**: add a tiny $\epsilon \mathbf{I}$ (e.g. $1e-6 \sigma^2$) to the diagonal. GPyTorch adds adaptive jitter automatically. Practically: prefer Matérn over RBF (better conditioned), keep

the sliding window from containing duplicate timestamps, and center/scale inputs.

6. The predictive covariance — our payload

Step 5 produces cov^* , the **predictive covariance** at the query point(s). For a single time it is a 3×3 matrix over (v_x, v_y, v_z) (with the multitask kernel of doc 02 §8).

This matrix is the deliverable we hand to the EKF as the **adaptive measurement-noise** `R` (doc 04). Key behaviours to expect:

- On/near a data time $\rightarrow$ small cov^* (denoised, trustworthy) $\rightarrow$ EKF weights it heavily.
- Inside a long dropout $\rightarrow$ cov^* inflates toward the prior $K(X^*, X^*) \rightarrow$ EKF down-weights it, smoothly handing authority back to IMU dead-reckoning. This is the graceful-degradation mechanism, falling straight out of the math — we don't hand-tune it.

7. Input/target preprocessing (small but important)

- **Center time** within the window (subtract the window's start) to keep numbers small and well-conditioned.

- **Standardize targets** (subtract mean, divide by std per axis) so default hyperparameter priors

behave; undo the transform on output.

- **Sort by time** and drop exact-duplicate timestamps (the pipeline already orders events by time,

see `implementation/01`).

8. From batch to streaming

Exact GP regression is a batch operation (refit on the current window). For a stream we either:

- **Refit periodically** on the sliding window (every `refit_every` samples) — simple, used for the

demo; hyperparameters change slowly so we don't refit every sample.

- **Update recursively** — which, for a Markovian kernel, is the Kalman filter

(doc 05). This is the elegant streaming/edge form.

Key takeaways

- Exact GP regression = build K_y , Cholesky L , solve for α , then mean = $K(X^*, X)\alpha$ and covariance = $K(X^*, X^*) - v^T v$.
- Cost is **$O(n^3)$ time / $O(n^2)$ memory** — the central constraint; we control it with a sliding window now and the Kalman form later.
- Per-sample noise lets us pass the DVL's own covariance in; **jitter** keeps Cholesky stable.
- The **predictive covariance** is the product we feed the EKF, and its growth during gaps gives graceful degradation for free.

Further reading

- Rasmussen & Williams, GPML, Algorithm 2.1 (the exact recipe above).
- GPyTorch docs — exact GP regression tutorial.
- Next: 04 — GPs for navigation and sensor fusion.